Software Engineering Assignment #03: Smart Campus Room Availability Checker

Student:Zhumash Assylbek

Student ID:24B031801

AI:Claude

Model: Sonnet 5.5

Stage A: Plan

> I am working on Software Engineering Assignment #03: Smart Campus Room Availability Checker.
>
> Before generating any code, I need a concise engineering plan (Stage A: Plan) for implementing `can_book(start, end, now, blocked, existing)` in `booking.py` and unit tests in `test_booking.py` using only the Python 3 standard library.
>
> Fixed Acceptance Criteria:
>
> - AC1: Candidate times are integer minutes after midnight: 0 <= start < end <= 1440; `start` must be strictly greater than `now` (assume `now` is valid in 0..1439).
> - AC2: Duration (`end - start`) must not exceed 120 minutes. Exactly 120 minutes is permitted.
> - AC3: Reject every request when `blocked` is True.
> - AC4: Reject overlap with any interval `(s, e)` in `existing`. Intervals touching at an endpoint (e.g., 600-660 and 660-720) do NOT overlap.
> - AC5: Return True only when valid, False otherwise, and do NOT mutate `existing`.
>
> Please provide ONLY:
>
> 1. A step-by-step validation plan for `can_book`.
> 2. Explicit assumptions (including Python type-checking nuances).
> 3. A list of boundary and edge test cases to include in `test_booking.py`.
>
> Do not generate the final implementation code yet.

Output

Stage A: Engineering Plan for `can_book(start, end, now, blocked, existing)`

1. Step-by-step validation plan

Checks run in a fixed order and short-circuit to `False` on the first failure. Cheap checks come first, and `existing` is only scanned once everything else has passed.

1. **Type validation (AC1, AC5).** Reject if `start`, `end` or `now` is not a true `int`. This includes rejecting `bool` (see assumptions). If `blocked` is not a `bool`, treat it as invalid.
2. **Blocked check (AC3).** If `blocked is True`, return `False`. This comes early because it overrides every other condition.
3. **Range and order check (AC1).** Require `0 <= start < end <= 1440`. This also guarantees `end - start >= 1`.
4. **Future-start check (AC1).** Require `start > now` (strict). `start == now` is rejected.
5. **Duration check (AC2).** Require `end - start <= 120`. Exactly 120 passes.
6. **Overlap check (AC4).** For each `(s, e)` in `existing`, the candidate conflicts if `start < e and s < end`. Strict inequalities make touching endpoints (`e == start` or `s == end`) non-overlapping. Return `False` on the first conflict.
7. **Success (AC5).** Return `True`.

The function is read-only with respect to `existing`. It iterates over it but never calls `sort`, `append`, `remove` or any other mutating method, and it does not reassign or alias it into something that gets modified.

2. Explicit assumptions

Inputs

- `now` is valid (0..1439), as stated in the assignment, so it is not re-validated for range.

- `existing` is an iterable of `(s, e)` pairs. Each pair is a 2-element tuple or list of ints with `s < e`, and the intervals are mutually valid.
- `existing` may be empty (and `None` is not a supported value).
- Existing intervals are not assumed to be sorted or non-overlapping with each other, so a full scan is used instead of binary search.

Python type-checking nuances

- `bool` is a subclass of `int`, so `isinstance(True, int)` is `True`. If the plan is to treat booleans as invalid times, the check must be `type(x) is int` or `isinstance(x, int) and not isinstance(x, bool)`. Otherwise `can_book(True, 2, 0, ...)` would be treated as `start=1`.
- Floats such as `600.0` are rejected, since AC1 requires integers. Strings and `None` are likewise rejected.
- `blocked` is checked with `blocked is True` (or `bool` type check), not truthiness, so that `1` or `"yes"` don't silently count as blocked. Decision to document: non-bool `blocked` returns `False` as invalid input.
- Invalid input returns `False` rather than raising an exception, consistent with AC5 ("False otherwise").
- Mixed comparisons like `None < 5` raise `TypeError` in Python 3, which is another reason type checks run before any comparison.

Design

- Single pure function with no global state, no I/O, standard library only (`unittest` for tests).
- Intervals are half-open `[start, end)`, which is what makes touching endpoints legal.

3. Boundary and edge test cases for `test_booking.py`

Use `unittest`. Suggested grouping by acceptance criterion:

Baseline

- Valid booking, empty `existing`, not blocked returns `True`.

AC1: range and ordering

- `start = 0` with `now` valid: `start > now` fails for any `now >= 0`, so test `False` (this shows the interaction between the lower bound and the future rule).
- `start = 1` with `now = 0` returns `True` (smallest valid start).
- `end = 1440` with valid start returns `True` (upper bound inclusive), e.g. `(1380, 1440, now=0)`.
- `end = 1441` returns `False`.
- `start = -1` returns `False`.
- `start == end` returns `False`.
- `start > end` returns `False`.
- `start = 1439, end = 1440` with `now = 1438` returns `True` (1-minute booking at day end).
- `now = 1439` with any valid `start` returns `False` (nothing can be strictly later and still within range except none; confirms no valid slot).

AC1: strict future start

- `start = now` returns `False`.
- `start = now + 1` returns `True`.
- `start = now - 1` returns `False`.

AC2: duration

- Duration 119 returns `True`.
- Duration 120 returns `True`.

- Duration 121 returns `False`.
- Duration 1 returns `True`.

AC3: blocked

- `blocked=True` with otherwise perfectly valid request and empty `existing` returns `False`.
- `blocked=True` with an otherwise invalid request returns `False`.
- `blocked=False` with valid request returns `True`.

AC4: overlap

- Existing `(600, 660)`, candidate `(600, 660)` (identical) returns `False`.
- Candidate fully inside existing, e.g. `(610, 650)` returns `False`.
- Candidate fully containing existing, e.g. `(590, 670)` returns `False`.
- Partial overlap at start, e.g. `(570, 630)` returns `False`.
- Partial overlap at end, e.g. `(630, 690)` returns `False`.
- Touching: existing `(600, 660)`, candidate `(660, 720)` returns `True`.
- Touching: existing `(660, 720)`, candidate `(600, 660)` returns `True`.
- One-minute overlap at either edge, e.g. candidate `(599, 601)` or `(659, 661)` returns `False`.
- Candidate sandwiched exactly between two existing intervals `(540, 600)` and `(660, 720)` with candidate `(600, 660)` returns `True`.
- Conflict only with the last item of a multi-item `existing` list returns `False` (ensures the full scan).
- Conflict with an unsorted `existing` list returns `False`.
- Multiple non-overlapping existing intervals elsewhere returns `True`.
- Empty `existing` returns `True`.

AC5: return type and non-mutation

- Return value is a `bool` (use `assertIs(result, True/False)`).
- Take a copy of `existing` before the call (`deepcopy` or `list(...)`), call `can_book` for both a `True` outcome and a `False` outcome, and assert the list is unchanged (same contents and order, including an unsorted list).
- If a tuple of tuples is used for `existing`, confirm it works without error.

Type and invalid-input edge cases

- `start` as float (`600.0`), string, or `None` returns `False`.
- `bool` passed as `start`, `end` or `now` (e.g. `True`) returns `False`.
- Non-bool `blocked` (e.g. `1`, `"yes"`) returns `False` (documented behavior).

Combined and interaction cases

- Valid duration and no overlap, but `start <= now` returns `False`.
- No overlap, but duration 121 returns `False`.
- Blocked plus overlap plus bad duration returns `False` (all failures at once).

Once you confirm or adjust these assumptions (notably the `bool` and non-bool `blocked` handling, and returning `False` rather than raising on bad types), I can move on to Stage B and generate the implementation and tests.

Stage B: Generate

> I approve this plan. Now proceed to Stage B (Generate):
>
> 1. Provide the complete Python 3 implementation of `booking.py` containing `can_book(start, end, now, blocked, existing)` using only the standard library.
> 2. Provide the complete `test_booking.py` using `unittest` covering all required test cases:
>
>> - Base data: `now = 540`, `blocked = False`, `existing = [(600, 660)]`
>> - Touching: 660-720 -> True
>> - Overlap: 630-690 -> False
>> - Blocked: 660-720 with `blocked = True` -> False
>> - Exactly 120 min: 720-840 -> True
>> - 121 min: 720-841 -> False
>> - Starts now: 540-570 -> False
>> - Reversed times (e.g., 700-680), equal start/end (700-700), day bounds (< 0 and > 1440, exact 1440 end), multiple existing bookings, and verification that `existing` is not mutated after function execution.
>
> 3. Explain each conditional branch in `can_book` and map it directly to AC1-AC5.

Output

I ran `python3 -m unittest -v` and all 40 tests pass. Both files, `booking.py` and `test_booking.py`, use only the standard library.

Branch-to-AC mapping for `can_book`

#	Branch	Returns `False` when	Maps to
1	`_is_int(start/end/now)`	Any of the three is not a true `int`, including `bool`, float, str or `None`.	AC1 (times must be integers), AC5 (return `False`, don't raise)
2	`type(blocked) is not bool`	`blocked` is not a real `bool`, e.g. `1` or `"yes"`.	AC3 and AC5; this is the documented non-bool decision from the plan
3	`if blocked`	`blocked` is `True`.	AC3
4	`0 <= start < end <= 1440` fails	`start < 0`, `end > 1440`, or `start >= end`, which covers reversed and zero-length bookings.	AC1
5	`start <= now`	The start is at or before the current time. It must be strictly later.	AC1
6	`end - start > 120`	The duration is over 120 minutes. Exactly 120 passes.	AC2
7	`start < e and s < end` inside the loop	The candidate overlaps any existing `(s, e)`. Strict `<` means `e == start` or `s == end` is touching, not overlap.	AC4
8	Final `return True`	Every check above passed.	AC5

Why this order:

- The type checks come first so that no comparison can raise a `TypeError`.
- The blocked check comes before the range and time checks because it overrides everything else.
- The `existing` scan comes last because it is the only step that costs more than constant time.

AC5 non-mutation: the function only iterates over `existing`. It never sorts, appends, removes or reassigns it. Three tests snapshot the list with `deepcopy` and confirm it is unchanged after `True`, `False`, blocked and invalid calls, including an unsorted list.

Assumptions to keep in mind:

- `existing` must be an iterable of `(s, e)` pairs. Passing `None` raises `TypeError`, as stated in the plan.
- `now` is trusted to be in 0..1439 and is only type-checked.
- Several tests set `now` and `existing` explicitly to isolate one rule. The base data (`now=540`, `blocked=False`, `[(600, 660)]`) applies by default.

Stage C: Critique и Stage D: Repair

Now proceed to Stage C (Critique) and Stage D (Repair) for `booking.py`:

1. Critically review our v1 implementation of `can_book` against AC1-AC5. Specifically check:
   - Does v1 properly reject boolean inputs for `start`, `end`, or `now` (since `isinstance(True, int)` is True in Python) or non-boolean values for `blocked`?
   - Does v1 handle malformed tuples or non-integer values inside `existing` safely without mutating the input?
2. Propose a small, targeted preventive cleanup (v2) of `booking.py` and add corresponding unit tests in `test_booking.py` without expanding the scope.
3. Explicitly list what changed between v1 and v2.

Output

Before judging v1, I'll run it against the inputs you asked about so the critique rests on observed behavior.

All 50 tests pass on v2 (40 from v1 plus 10 new). The nonzero return code in that run came from diff, which exits 1 whenever files differ, not from the tests.

Stage C: Critique of v1

I ran v1 against the inputs you listed instead of reasoning from the code alone.

1. Booleans and non-bool blocked: v1 handles these correctly.

- **can_book(True, 5, 0, ...), end=True and now=False all return False, because _is_int uses type(value) is int instead of isinstance.**
- **blocked=1 returns False because of the type(blocked) is not bool check.**
- **No repair was needed here.**

2. Malformed existing: v1 has a real gap, in two forms.

Input in existing	v1 behavior	Problem
None, 5 (non-iterable)	raises TypeError	Violates AC5, which says False otherwise
(600,), (600, 660, 720), "abc"	raises ValueError	Same
None, "ab", (600, "x") as an entry	raises TypeError	Same
(True, 5)	returns True, silently treating it as the interval (1, 5)	Silent acceptance of garbage, inconsistent with the bool rejection for start/end/now
(600.5, 660.5)	returns False, but only by accident of the comparison	Right answer for the wrong reason

The raising cases are the main finding. v1 already returns False instead of raising for bad start, end, now and blocked, but not for bad existing.

Non-mutation is not affected. v1 only iterates existing, so no input, malformed or not, causes a mutation.

One test-quality issue in v1. In test_existing_not_mutated_on_true, I had a stray call with start 760 and end 760. It returned False because the interval was zero-length, not for the reason the test implied. I removed it.

Stage D: v2 repair

What changed in booking.py:

1. **New helper _is_valid_interval(item). It accepts only a tuple or list of exactly 2 elements, both real ints. The same type(x) is int rule is used, so bools and floats are rejected.**
2. **iter(existing) wrapped in try/except TypeError. A non-iterable existing now returns False.**
3. **The for s, e in existing: loop became for item in intervals:. Each item is validated before it is unpacked, and a malformed item returns False. The overlap test start < e and s < end is unchanged.**
4. **Docstring updated to state the fail-closed contract.**

What did not change: the order and logic of every other check (AC1 to AC3, duration, blocked), the return values for all well-formed input, and the single-pass read-only scan.

Why return False instead of raising? If existing cannot be read reliably, availability cannot be confirmed, so refusing the booking is the safe outcome. It also matches AC5 and the way v1 already treats the other inputs.

What changed in test_booking.py:

- **Fixed: the misleading stray call in the non-mutation test, as noted above.**
- **Added TestMalformedExisting, 10 tests:**
  - **Non-iterable existing.**
  - **Wrong-length entries.**
  - **Non-sequence entries and strings.**
  - **Non-int values inside entries.**
  - **Bool values inside entries (the (True, 5) case).**
  - **A malformed entry placed after a valid one.**
  - **A sanity check that the same request is True when existing is well formed.**
  - **Lists as entries and generator input still work.**
  - **No mutation of a malformed existing.**

Deliberate non-changes, to keep scope small:

- **v2 does not check s < e or the 0..1440 range of existing intervals. The plan assumed existing intervals are valid, and no AC asks for more. A reversed existing interval like (660, 600) is still treated as well-formed.**
- **v2 does not materialize existing, so one-shot iterators such as generators still work.**

Bolt prompt

Build a responsive one-page web prototype called "Smart Campus Room Availability Checker" for a single room ("Room 304 — Collaborative Study Lab") using fictional in-memory data only. STRICT SCOPE & CONSTRAINTS:

- The page checks availability ONLY; it must NOT create, confirm, or add bookings to the existing list.
- Explicitly FORBID login/authentication, database, backend server, external APIs, payments, and multi-room selection. REQUIRED ACCEPTANCE CRITERIA (AC1-AC6):
- Base fictional state:
  - now defaults to 540 (09:00), with an input/control allowing the user to adjust now (valid integer 0..1439) for testing.
  - blocked defaults to False, with a toggle switch to simulate blocking/unblocking the room.
  - existing bookings list is fixed to [(600, 660)] (10:00 - 11:00), plus an option to switch to a multi-booking test preset [(600, 660), (900, 960)] without mutating the underlying array.
- AC1: Candidate start and end times are entered as integer minutes after midnight (0 <= start < end <= 1440), and start must be strictly greater than now (start > now).

- AC2: Duration (end - start) must not exceed 120 minutes (120 minutes is allowed; 121 is rejected).
- AC3: Reject every request immediately when blocked is True.
- AC4: Reject any overlap with existing bookings (start < existingEnd && end > existingStart). Intervals touching at an endpoint (e.g., 660-720 when 600-660 is booked) do NOT overlap and must be allowed.
- AC5: Display an unambiguous "Available" or "Unavailable" badge/result and never mutate the existing bookings list when checking availability.
- AC6: Provide clear, specific validation feedback explaining which rule (AC1, AC2, AC3, or AC4) caused the rejection, and ensure a clean, usable layout on narrow/mobile screens (375px width).

Output:

I'll build this as a single-page app with in-memory state only (no Bolt Database needed since the spec explicitly forbids persistence). Let me create it.

0123456789

actions taken

Build passes. I built a single-page Room Availability Checker for "Room 304 — Collaborative Study Lab" using only in-memory data — no login, database, backend, or bookings are ever created.

What's on the page:

- **Simulation controls**: a slider + number input to set "now" (0–1439, defaults to 09:00), a toggle switch to block/unblock the room, and a two-button preset selector for the existing bookings (Single [10:00–11:00] or Multi adding [15:00–16:00]) — switching presets never mutates the underlying list.
- **Request inputs**: start and end as integer minutes after midnight, with a live duration readout.
- **Result badge**: a clear "Available" or "Unavailable" panel. When rejected, each failing rule is listed with its own badge (AC1/AC2/AC3/AC4) and a plain-language explanation — e.g. "Start must be strictly after now", "Duration is 135 minutes; max is 120", "Room is blocked", or "Overlaps existing booking 10:00–11:00. Touching endpoints are allowed."
- **Day timeline**: a visual bar showing existing bookings, your requested slot (green when available, red when rejected, grey when blocked), and a "now" marker.
- **Rules reference** section at the bottom.

All six acceptance criteria are enforced: start>now and 0≤start<end≤1440 (AC1), duration≤120 (AC2), instant reject when blocked (AC3), overlap rejection with endpoint-touching allowed (AC4), unambiguous Available/Unavailable badge with no mutation of the existing list (AC5), and rule-specific feedback on a responsive layout that fits 375px screens (AC6).

Bolt input v2:

Perform a targeted cleanup (v1 -> v2) to fix two specific issues without adding any out-of-scope features (do not add booking creation, backend, or login):

1. Functional Input Validation Defect (AC1): HTML number inputs allow decimal values (e.g., 660.5) or empty/NaN strings. Enforce strict integer parsing and validation (Number.isInteger) on raw input strings before evaluating AC1-AC4, and show an explicit AC1 validation message if a decimal or non-integer value is entered.
2. Usability & Mobile Layout Improvement (AC6):
    - Next to every minute input (now, start, end, and duration), dynamically display the human-readable HH:MM time equivalent (e.g., 660 min (11:00 AM)) so users do not have to calculate minutes from midnight mentally.
    - Improve the narrow/mobile layout (< 400px) by stacking the time inputs and status banner compactly with clear visual spacing, and add a "Quick Test Presets" bar (which only fills the

input fields without mutating existing bookings) for rapid verification of boundary cases (660-720 Touching, 630-690 Overlap, 720-840 120m, 720-841 121m, 540-570 Starts Now).

Output

Build and typecheck both pass. Two targeted fixes applied, nothing out of scope added:

1. Strict integer validation (AC1): Replaced the lenient Number.parseInt parsing with a strict parseIntegerInput helper that rejects decimals (like 660.5), blanks, and non-numeric strings by returning null. checkAvailability now treats non-integer start/end as an explicit AC1 failure with the message: "Start and end must be whole numbers of minutes after midnight — decimals, blank, or non-numeric values are not allowed." The start/end inputs are now type="text" with inputMode="numeric" so decimals can actually be typed and caught (HTML number inputs silently strip them). Invalid values get a red border and inline "Not a whole number — decimals and text are rejected (AC1)" hint.

2. Usability & mobile (AC6):

- Every minute input now shows its human-readable equivalent: now displays 540 min · 9:00 AM · range 0–1439, start/end show 660 min (11:00 AM), and the duration readout shows 60 min (1:00).
- Added a **Quick Test Presets** bar with five boundary-case buttons (660–720 Touching, 630–690 Overlap, 720–840 120 min, 720–841 121 min, 540–570 Starts Now). Each only fills the input fields — it never mutates the existing bookings list.
- Tightened the narrow/mobile layout: the result banner now stacks icon-over-text below 640px, the now-control hint wraps cleanly, and inputs stack full-width on phones (375px) while remaining side-by-side on wider screens.